Notes on Examples and Proofs

1 Perceptron: XOR and the Limitation of Linear Separability

The perceptron is a single-layer linear classifier. Given an input $x \in \mathbb{R}^d$, it computes $f(x) = \operatorname{sgn}(w^\top x + b)$, where w is the weight vector and b is the bias term. The decision boundary is the hyperplane $w^\top x + b = 0$. Therefore, the perceptron can only separate data by a linear boundary.

A dataset $\{(x_i, y_i)\}_{i=1}^n$, where $y_i \in \{-1, 1\}$, is linearly separable if there exist w and b such that $y_i(w^\top x_i + b) > 0$ for every i. This condition says that every positive point lies on one side of the hyperplane and every negative point lies on the other side.

XOR Example

Consider the XOR classification problem:

x	y
$(0,0)$	-1
$(1,0)$	1
$(0,1)$	1
$(1,1)$	-1

The two positive points are $(1,0)$ and $(0,1)$, while the two negative points are $(0,0)$ and $(1,1)$. Intuitively, this cannot be separated by one straight line: the positive points lie on opposite corners, and the negative points lie on the other opposite corners.

Proof that XOR is not linearly separable

Suppose, for contradiction, that XOR is linearly separable. Then there exist w_1, w_2 and b such that the perceptron $f(x) = \operatorname{sgn}(w_1 x_1 + w_2 x_2 + b)$ correctly classifies all four points.

Because $(0,0)$ has label -1, we need $b < 0$.

Because $(1,0)$ has label 1, we need $w_1 + b > 0$. Therefore $w_1 > -b$.

Because $(0,1)$ has label 1, we need $w_2 + b > 0$. Therefore $w_2 > -b$.

Adding the last two inequalities gives $w_1 + w_2 > -2b$. Since $b < 0$, this implies $w_1 + w_2 + b > -b > 0$.

But $(1,1)$ has label -1, so correct classification requires $w_1 + w_2 + b < 0$.

Thus we have both $w_1 + w_2 + b > 0$ and $w_1 + w_2 + b < 0$, which is impossible. Therefore, XOR is not linearly separable, and a single perceptron cannot solve XOR.

Why this matters

This proof shows the exact mathematical limitation of the perceptron. The problem is not simply that the perceptron is "too small." The problem is structural: its decision boundary is linear. Therefore, it cannot represent nonlinear classification patterns such as XOR. This motivates the move from a single-layer perceptron to a multi-layer perceptron.

2 Multi-Layer Perceptron: Solving XOR with a Hidden Layer

A multi-layer perceptron, or MLP, adds hidden layers and nonlinear activation functions. Instead of directly applying one linear classifier to the original input, an MLP first transforms the input into a new representation. A simple one-hidden-layer network has the form $h = \sigma(W_1 x + b_1)$ and $\hat{y} = \sigma(W_2 h + b_2)$, where h is the hidden representation and σ is a nonlinear activation function.

The key idea is that the hidden layer can transform a nonlinear problem in the original input space into a linearly separable problem in a new feature space.

Constructive example: an MLP solves XOR

Let $x_1, x_2 \in \{0, 1\}$. The XOR function is 1 when exactly one of x_1 and x_2 is equal to 1, and is 0 otherwise.

We construct two hidden units. The first hidden unit detects whether at least one input is active:

$$h_1 = \mathbf{1}\{x_1 + x_2 \geq 1\}$$

The second hidden unit detects whether both inputs are active:

$$h_2 = \mathbf{1}\{x_1 + x_2 \geq 2\}$$

Then XOR can be represented as "at least one input is active, but not both." So define the output as

$$\hat{y} = \mathbf{1}\{h_1 - h_2 \geq 1\}$$

Now check all four cases:

(x_1, x_2)	$h_1 = \mathbf{1}\{x_1 + x_2 \geq 1\}$	$h_2 = \mathbf{1}\{x_1 + x_2 \geq 2\}$	$\hat{y} = \mathbf{1}\{h_1 - h_2 \geq 1\}$
$(0, 0)$	0	0	0
$(1, 0)$	1	0	1
$(0, 1)$	1	0	1
$(1, 1)$	1	1	0

This exactly matches XOR.

Why the hidden layer works

The original four points are not linearly separable in the (x_1, x_2) plane. But the hidden layer maps each input into a new representation (h_1, h_2).

The transformed points are:

(x_1, x_2)	(h_1, h_2)	XOR output
$(0,0)$	$(0,0)$	0
$(1,0)$	$(1,0)$	1
$(0,1)$	$(1,0)$	1
$(1,1)$	$(1,1)$	0

In the hidden representation, the positive case is represented by $(1,0)$, while the negative cases are represented by $(0,0)$ and $(1,1)$. The output unit can now separate the cases by checking whether $h_1 - h_2 \geq 1$.

Why this matters

The MLP solves the failure of the perceptron by learning intermediate features. The perceptron asks whether the original data are linearly separable. The MLP instead asks whether the data can be transformed into a representation where prediction becomes easier. This is the beginning of representation learning.

3 Backpropagation: Chain Rule Derivation for One Hidden Layer

The MLP is more expressive than the perceptron, but this creates a new problem: how do we train the hidden-layer weights? Backpropagation answers this question using the chain rule.

Consider a one-hidden-layer neural network. Let the input be $x \in \mathbb{R}^d$. The hidden pre-activation is $z^{(1)} = W^{(1)}x + b^{(1)}$. The hidden activation is $h = \sigma(z^{(1)})$. The output pre-activation is $z^{(2)} = W^{(2)}h + b^{(2)}$. The prediction is $\hat{y} = g(z^{(2)})$. The loss is $L(\hat{y}, y)$.

The goal is to compute gradients such as $\partial L/\partial W^{(2)}$ and $\partial L/\partial W^{(1)}$.

Output-layer gradient

Define the output-layer error term as $\delta^{(2)} = \partial L/\partial z^{(2)}$. Since $z^{(2)} = W^{(2)}h + b^{(2)}$, each output-layer weight affects the loss through $z^{(2)}$.

Using the chain rule, we get

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} h^\top$$

and

$$\frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}$$

This says that the gradient for the output weights is the output error times the hidden activation.

Hidden-layer gradient

The hidden-layer weights are more complicated because they affect the loss indirectly. The path is $W^{(1)} \to z^{(1)} \to h \to z^{(2)} \to \hat{y} \to L$.

By the chain rule, the hidden-layer error term is

$$\delta^{(1)} = \frac{\partial L}{\partial z^{(1)}} = \left((W^{(2)})^\top \delta^{(2)} \right) \odot \sigma'(z^{(1)})$$

Here, $(W^{(2)})^\top \delta^{(2)}$ sends the output-layer error backward into the hidden layer, and $\sigma'(z^{(1)})$ accounts for the derivative of the hidden activation function. The symbol $\odot$ means element-wise multiplication.

Since $z^{(1)} = W^{(1)}x + b^{(1)}$, we then get

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^\top$$

and

$$\frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}$$

Gradient descent update

Once these gradients are computed, the network parameters can be updated by gradient descent. For layer ℓ, the update is

$$W^{(\ell)} \leftarrow W^{(\ell)} - \eta \frac{\partial L}{\partial W^{(\ell)}}$$

and

$$b^{(\ell)} \leftarrow b^{(\ell)} - \eta \frac{\partial L}{\partial b^{(\ell)}}$$

where $\eta > 0$ is the learning rate.

Why this is a proof of trainability

The derivation shows that every hidden-layer weight has a computable gradient. Even though the hidden weights do not directly appear in the final loss, the chain rule traces their effect through the network. Therefore, backpropagation makes it possible to train hidden layers systematically rather than manually adjusting them.

The key logic is:

forward pass computes predictions

loss measures error

chain rule sends error backward

gradient descent updates every layer

This is why backpropagation is one of the central mathematical tools behind modern neural networks.

4 CNN: Convolution and Parameter Sharing

Convolutional neural networks, or CNNs, were developed mainly for image-like data. Their statistical advantage comes from two assumptions: local structure and parameter sharing. Images have local patterns, such as edges and corners, and the same kind of pattern can appear in many locations. CNNs build this structure directly into the model.

Convolution operation

Let X be a two-dimensional image and let K be an $m \times n$ filter. The convolution output at location (i, j) is

$$Y_{i,j} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} K_{a,b} X_{i+a,j+b}$$

This means the same filter K is applied to every local image patch. For example, a vertical-edge filter may be

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

This filter compares the left side of a small patch with the right side. If the left and right sides are very different, the output has large magnitude, indicating a possible vertical edge.

Parameter sharing example

Suppose an image has size 100×100, so it has 10,000 input pixels. A fully connected layer with 1,000 hidden units would require $10{,}000 \times 1{,}000 = 10{,}000{,}000$ weights, not including biases.

Now compare this with a CNN using one 3×3 filter. The filter has only $3 \times 3 = 9$ weights. Even though the filter is applied across many image locations, the same 9 weights are reused everywhere. This is parameter sharing.

If the CNN uses 64 filters of size 3×3, then the number of filter weights is only $64 \times 9 = 576$. This is far smaller than 10,000,000.

Why CNNs are statistically efficient

The statistical reason CNNs work well is that they reduce the effective number of parameters. With fewer parameters, the model usually needs less data to estimate them reliably. This can reduce overfitting.

The basic comparison is:

fully connected layer: many independent weights

convolutional layer: small filter reused across space

Therefore, CNNs impose a useful inductive bias: the same visual pattern can matter regardless of where it appears in the image.

Why this matters

CNNs show that neural network architecture should reflect the structure of the data. The perceptron and MLP are general-purpose function approximators, but they do not automatically exploit spatial locality. CNNs improve prediction by building in assumptions that are true for images: nearby pixels are related, and patterns repeat across locations.

5 RNN: Hidden-State Recurrence and Sequential Dependence

Feedforward neural networks process each input independently. But many data types are sequential. In language, speech, time series, and event data, the order of observations matters. Recurrent neural networks, or RNNs, introduce a hidden state that carries information forward over time.

Basic hidden-state recurrence

Given a sequence $x_1, x_2, \ldots, x_T$, an RNN computes

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b)$$

where h_t is the hidden state at time t, x_t is the current input, and h_{t-1} is the previous hidden state.

The output can be written as

$$\hat{y}_t = g(W_y h_t + c)$$

The key point is that h_t depends on both the current input x_t and the past summary h_{t-1}. Thus, h_t is not just a function of x_t. It is a function of the entire history $x_1, \ldots, x_t$.

Proof by expansion

To see this, write the recurrence explicitly. At time t, we have $h_t = \phi(W_x x_t + W_h h_{t-1} + b)$. But $h_{t-1} = \phi(W_x x_{t-1} + W_h h_{t-2} + b)$. Substituting this into the first equation gives

$$h_t = \phi\left(W_x x_t + W_h \phi(W_x x_{t-1} + W_h h_{t-2} + b) + b\right)$$

Substituting again for h_{t-2} shows that h_t depends on x_t, x_{t-1}, x_{t-2}, and so on. Continuing this expansion gives

$$h_t = F_t(x_1, x_2, \ldots, x_t)$$

for some function F_t determined by the RNN parameters. Therefore, the hidden state is a learned summary of the sequence history.

Example: sentiment depends on word order

Consider the sentence:

"The movie was not good."

A model that ignores order may see the word "good" and predict positive sentiment. But an RNN processes words sequentially. When it reaches "good," its hidden state can still contain information from the earlier word "not." Therefore, the model can learn that "not good" has a different meaning from "good."

The relevant sequence is:

$$\text{The} \to \text{movie} \to \text{was} \to \text{not} \to \text{good}$$

At each step, the hidden state updates. By the final word, h_t contains a compressed representation of the previous words. This is why recurrence is useful for language and time-series prediction.

Why this matters

RNNs extend neural networks from fixed inputs to ordered sequences. The MLP learns a representation of one input. The RNN learns a representation of a history. This marks an important shift from static prediction to sequential prediction.

The conceptual transition is:

$$\text{prediction from one input} \quad \longrightarrow \quad \text{prediction from accumulated context}$$

This sets up the later problem: although RNNs can represent sequence history in principle, they often struggle to learn long-range dependence in practice because gradients must pass through many repeated transformations.

6 LSTM: Gated Memory and Long-Range Dependence

A vanilla RNN updates its hidden state by $h_t = \phi(W_x x_t + W_h h_{t-1} + b)$. This means all memory must pass through the same repeated nonlinear transformation. As a result, old information is easily overwritten or weakened.

The long short-term memory network, or LSTM, changes this by introducing a separate memory channel called the cell state. At each time t, the LSTM has both a hidden state h_t and a cell state

c_t. The hidden state is the visible output of the recurrent unit, while the cell state is the internal memory.

LSTM gate equations

Given input x_t, previous hidden state h_{t-1}, and previous cell state c_{t-1}, the LSTM computes four main quantities.

The forget gate is $f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$. It decides how much of the old memory c_{t-1} to keep.

The input gate is $i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$. It decides how much new information to write into memory.

The candidate memory is $\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$. It contains the new information that could be added to the cell state.

The output gate is $o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$. It decides how much of the internal memory should be exposed as the hidden state.

The central LSTM memory update is

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

and the hidden state is

$$h_t = o_t \odot \tanh(c_t)$$

where $\odot$ denotes element-wise multiplication.

The most important equation is $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. It says that the new memory equals retained old memory plus selected new memory.

Memory-cell intuition

The forget gate f_t acts like a memory-preservation switch. If one component of f_t is close to 1, then the corresponding component of c_{t-1} is mostly copied into c_t. If it is close to 0, then that old memory is mostly erased.

The input gate i_t acts like a writing switch. If one component of i_t is close to 1, then the new candidate information $\tilde{c}_t$ is written strongly into memory. If it is close to 0, the candidate information is mostly ignored.

Thus the LSTM can learn rules of the form:

keep important old information forget irrelevant old information write useful new information

This is why LSTMs are better suited than vanilla RNNs for long-range dependence.

Proof sketch: why LSTM helps with vanishing gradients

The key difference between a vanilla RNN and an LSTM is the cell-state path. In a vanilla RNN, the hidden state is repeatedly transformed through nonlinear functions. In an LSTM, the cell state has an additive update.

From $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, we have, componentwise,

$$c_{t,j} = f_{t,j} c_{t-1,j} + i_{t,j} \tilde{c}_{t,j}$$

If we focus on the direct memory path and treat f_t, i_t, and $\tilde{c}_t$ as fixed for the purpose of seeing the main gradient route, then

$$\frac{\partial c_{t,j}}{\partial c_{t-1,j}} = f_{t,j}$$

Across many time steps,

$$\frac{\partial c_{T,j}}{\partial c_{t,j}} = \prod_{k=t+1}^{T} f_{k,j}$$

This shows exactly how information is preserved. If the forget gates satisfy $f_{k,j} \approx 1$ for the relevant memory component, then $\prod_{k=t+1}^{T} f_{k,j} \approx 1$, so the gradient does not quickly vanish along that memory path. By contrast, if $f_{k,j}$ is much smaller than 1, the product shrinks and the memory is intentionally forgotten.

So the LSTM does not magically remove all gradient problems. Rather, it gives the network a learnable way to control the gradient path. When long-term memory is useful, it can keep f_t near 1. When old information is irrelevant, it can push f_t closer to 0.

Example: remembering subject-verb agreement

Consider the sentence:

"The book that the students borrowed from the library was interesting."

The correct verb is "was" because the main subject is "book." However, many words appear between "book" and "was." A vanilla RNN must carry information about "book" through every intermediate word. An LSTM can store the singular subject information in the cell state.

At the word "book," the input gate may write subject-number information into memory. Across the phrase "that the students borrowed from the library," the forget gate may keep this information by staying close to 1. When the model reaches "was," the output gate can expose the relevant part of the cell state to help predict the correct verb.

In simplified form, the memory behavior is:

$$c_{\text{book}} \approx \text{stores singular subject}$$

$$f_t \approx 1 \quad \text{for intermediate words}$$

$$c_{\text{was}} \approx c_{\text{book}}$$

Therefore, the model can preserve information that matters across a long sentence.

7 GRU: A Simpler Gated Recurrent Unit

The gated recurrent unit, or GRU, is a simpler gated alternative to the LSTM. It does not have a separate cell state c_t. Instead, the hidden state h_t itself serves as the memory.

The GRU uses two main gates: the update gate and the reset gate.

GRU gate equations

Given input x_t and previous hidden state h_{t-1}, the update gate is

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

The reset gate is

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

The final hidden state is

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

The most important equation is $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. It says that the new hidden state is a weighted combination of old memory and new candidate memory.

Interpretation of the update gate

The update gate z_t controls how much the hidden state changes.

If $z_t \approx 0$, then $h_t \approx h_{t-1}$. The model mostly keeps the old memory.

If $z_t \approx 1$, then $h_t \approx \tilde{h}_t$. The model mostly replaces the old memory with new information.

So z_t plays a role similar to combining the forget and input gates in an LSTM. It decides how much to preserve and how much to update.

Interpretation of the reset gate

The reset gate r_t controls how much past information is used when forming the candidate state $\tilde{h}_t$. If $r_t \approx 0$, then $r_t \odot h_{t-1} \approx 0$, so the candidate state mostly ignores the past. If $r_t \approx 1$, then the

candidate state uses the previous hidden state more fully.

Thus the reset gate can help the model begin a new memory segment. For example, if a sentence changes topic, the GRU may reduce r_t so that old information has less influence on the new candidate state.

Proof sketch: GRU creates a direct memory path

The GRU hidden-state update is $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. Componentwise, this is

$$h_{t,j} = (1 - z_{t,j})h_{t-1,j} + z_{t,j}\tilde{h}_{t,j}$$

Again focusing on the direct path from $h_{t-1,j}$ to $h_{t,j}$, we get

$$\frac{\partial h_{t,j}}{\partial h_{t-1,j}} \approx 1 - z_{t,j}$$

Across many time steps,

$$\frac{\partial h_{T,j}}{\partial h_{t,j}} \approx \prod_{k=t+1}^{T} (1 - z_{k,j})$$

If $z_{k,j} \approx 0$, then $1 - z_{k,j} \approx 1$, so the hidden state can carry information forward with little decay. If $z_{k,j} \approx 1$, then the old information is replaced by the candidate state. Thus, like the LSTM, the GRU creates a learnable mechanism for preserving or replacing memory.

LSTM versus GRU

The LSTM has a separate cell state c_t and uses forget, input, and output gates. The GRU has no separate cell state and uses update and reset gates. The LSTM gives more explicit control over memory, while the GRU is simpler and usually has fewer parameters.

Feature	LSTM	GRU
Memory	c_t and h_t	h_t only
Main gates	f_t, i_t, o_t	z_t, r_t
Parameter count	larger	smaller
Main idea	explicit cell memory	simplified gated memory

Both models solve the same basic problem: ordinary recurrence is too fragile for long-range dependence.

8 Word2Vec and Embeddings: Words as Continuous Vectors

Before embeddings, a common way to represent words was one-hot encoding. If the vocabulary has size V, each word is represented by a vector in $\mathbb{R}^V$ with one entry equal to 1 and all other entries equal to 0.

For example, suppose the vocabulary is

$$\{\text{king}, \text{queen}, \text{man}, \text{woman}, \text{apple}\}$$

Then one possible one-hot representation is

$$\text{king} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \qquad \text{queen} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

The problem is that one-hot vectors do not encode similarity. For any two different one-hot vectors e_i and e_j, we have $e_i^\top e_j = 0$. So "king" and "queen" are just as unrelated as "king" and "apple" under this representation.

Proof: one-hot vectors treat all different words as equally distant

Let e_i and e_j be two different one-hot vectors in $\mathbb{R}^V$. Then e_i has a 1 in position i and 0 elsewhere, while e_j has a 1 in position j and 0 elsewhere. If $i \neq j$, then they do not share any nonzero coordinate, so $e_i^\top e_j = 0$.

Their squared Euclidean distance is

$$\|e_i - e_j\|^2 = (e_i - e_j)^\top (e_i - e_j)$$

Expanding,

$$\|e_i - e_j\|^2 = e_i^\top e_i - 2e_i^\top e_j + e_j^\top e_j$$

Since $e_i^\top e_i = 1$, $e_j^\top e_j = 1$, and $e_i^\top e_j = 0$, we get

$$\|e_i - e_j\|^2 = 2$$

Therefore, $\|e_i - e_j\| = \sqrt{2}$ for every pair of different words. This proves that one-hot encoding makes every pair of distinct words equally far apart. It cannot represent that some words are semantically closer than others.

Dense embeddings

Word embeddings solve this by mapping each word to a dense vector $v_w \in \mathbb{R}^d$, where $d \ll V$. Instead of a sparse vocabulary-sized vector, each word gets a lower-dimensional learned representation.

The goal is that words appearing in similar contexts should have similar vectors. For example, the vectors v_{king} and v_{queen} should be more similar than v_{king} and v_{apple}.

Cosine similarity example

A standard way to measure embedding similarity is cosine similarity:

$$\cos(v_a, v_b) = \frac{v_a^\top v_b}{\|v_a\|\|v_b\|}$$

This measures the angle between two vectors. If $\cos(v_a, v_b)$ is close to 1, the vectors point in a similar direction. If it is close to 0, they are nearly orthogonal. If it is negative, they point in opposite directions.

For a simple numerical example, suppose

$$v_{\text{king}} = (0.8, 0.6), \qquad v_{\text{queen}} = (0.75, 0.65), \qquad v_{\text{apple}} = (-0.2, 0.9)$$

Then

$$v_{\text{king}}^\top v_{\text{queen}} = 0.8(0.75) + 0.6(0.65) = 0.99$$

Also, $\|v_{\text{king}}\| = \sqrt{0.8^2 + 0.6^2} = 1$, and $\|v_{\text{queen}}\| = \sqrt{0.75^2 + 0.65^2} \approx 0.992$. So

$$\cos(v_{\text{king}}, v_{\text{queen}}) \approx \frac{0.99}{1(0.992)} \approx 0.998$$

Now compare "king" and "apple":

$$v_{\text{king}}^\top v_{\text{apple}} = 0.8(-0.2) + 0.6(0.9) = 0.38$$

Since $\|v_{\text{apple}}\| = \sqrt{(-0.2)^2 + 0.9^2} \approx 0.922$, we get

$$\cos(v_{\text{king}}, v_{\text{apple}}) \approx \frac{0.38}{1(0.922)} \approx 0.412$$

Thus the embedding geometry says that "king" is closer to "queen" than to "apple."

Word2Vec skip-gram objective

Word2Vec learns embeddings from word context. The central idea is distributional: words that appear in similar contexts tend to have similar meanings.

In the skip-gram model, a center word w_t is used to predict nearby context words w_{t+j}. With window size m, the objective is

$$\max_\theta \sum_{t=1}^{T} \sum_{\substack{-m \le j \le m \\ j \ne 0}} \log P_\theta(w_{t+j} \mid w_t)$$

A common softmax form is

$$P(o \mid c) = \frac{\exp(v_o^\top v_c)}{\sum_{w \in V} \exp(v_w^\top v_c)}$$

where c is the center word, o is the context word, v_c is the center embedding, and v_o is the output embedding.

Why similar contexts create similar vectors

Suppose two words a and b often appear in similar contexts. Then during training, both v_a and v_b are pushed to assign high probability to many of the same context words. Since $P(o \mid c)$ increases when $v_o^\top v_c$ increases, both v_a and v_b are pushed to have large dot products with similar context vectors v_o. As a result, v_a and v_b tend to move in similar directions in embedding space.

This is the mathematical reason behind the slogan:

$$\text{similar context} \quad \Longrightarrow \quad \text{similar vector representation}$$

Negative sampling

Computing the full softmax is expensive because the denominator sums over the entire vocabulary V. Negative sampling replaces this with a simpler binary classification problem.

For a true pair (c, o), the model wants $v_o^\top v_c$ to be large. For negative samples $n_1, \ldots, n_K$, the model wants $v_{n_k}^\top v_c$ to be small.

The objective for one positive pair is

$$\log \sigma(v_o^\top v_c) + \sum_{k=1}^{K} \log \sigma(-v_{n_k}^\top v_c)$$

The first term pulls the center word toward the true context word. The second term pushes the center word away from randomly sampled non-context words.

Why this matters

Embeddings mark an important shift in language modeling. Words are no longer treated as isolated symbols. They become points in a learned geometric space. This makes it possible to represent similarity, analogy, and contextual relation statistically.

The transition is

$$\text{word as discrete symbol} \quad \longrightarrow \quad \text{word as vector} \quad \longrightarrow \quad \text{meaning as geometry}$$

Modern language models use contextual embeddings rather than static Word2Vec embeddings, but the core idea remains: language is represented through learned vectors.

9 Attention: Softmax Weighted Averages and Token Relevance

Attention was introduced to solve another limitation of recurrent models. In a basic sequence-to-sequence RNN, the encoder often compresses the whole input into one fixed-length vector. For long sequences, this creates an information bottleneck. Attention avoids this by letting the model look back at all input positions and assign different weights to different tokens.

Basic attention setup

Suppose the input sequence has representations $h_1, h_2, \ldots, h_T$. At a given output step, the model has a query vector q. For each input position i, it computes a relevance score $s_i = q^\top k_i$, where k_i is the key vector for position i.

The attention weights are computed using softmax:

$$\alpha_i = \frac{\exp(s_i)}{\sum_{j=1}^{T} \exp(s_j)}$$

The context vector is the weighted average

$$c = \sum_{i=1}^{T} \alpha_i v_i$$

where v_i is the value vector for position i.

Proof: attention weights form a probability distribution

For every i, $\exp(s_i) > 0$, and the denominator $\sum_{j=1}^{T} \exp(s_j)$ is positive. Therefore $\alpha_i > 0$ for every i.

Also,

$$\sum_{i=1}^{T} \alpha_i = \sum_{i=1}^{T} \frac{\exp(s_i)}{\sum_{j=1}^{T} \exp(s_j)}$$

Since the denominator is the same for all i, we can write

$$\sum_{i=1}^{T} \alpha_i = \frac{\sum_{i=1}^{T} \exp(s_i)}{\sum_{j=1}^{T} \exp(s_j)} = 1$$

Therefore the attention weights are nonnegative and sum to 1. This means attention computes a probability-like distribution over input positions.

Proof: the context vector is a convex combination

Because $\alpha_i \geq 0$ and $\sum_{i=1}^{T} \alpha_i = 1$, the context vector $c = \sum_{i=1}^{T} \alpha_i v_i$ is a convex combination of the value vectors $v_1, \ldots, v_T$. Therefore, c lies inside the convex hull of the value vectors.

This matters because attention does not arbitrarily create a context vector from nowhere. It builds the context vector by averaging available token information, with weights determined by relevance.

Numerical example: choosing the relevant token

Suppose a model is interpreting the sentence:

"The animal did not cross the street because it was tired."

To understand "it," the model should attend more to "animal" than to "street." Suppose the relevant scores for three candidate tokens are:

$$s_{\text{animal}} = 3, \qquad s_{\text{street}} = 1, \qquad s_{\text{tired}} = 0$$

The softmax weights are

$$\alpha_{\text{animal}} = \frac{e^3}{e^3 + e^1 + e^0}$$

$$\alpha_{\text{street}} = \frac{e^1}{e^3 + e^1 + e^0}$$

$$\alpha_{\text{tired}} = \frac{e^0}{e^3 + e^1 + e^0}$$

Since $e^3 \approx 20.09$, $e^1 \approx 2.72$, and $e^0 = 1$, the denominator is approximately 23.81. Thus

$$\alpha_{\text{animal}} \approx 0.844$$

$$\alpha_{\text{street}} \approx 0.114$$

$$\alpha_{\text{tired}} \approx 0.042$$

So the context vector places most of its weight on "animal." This is how attention can dynamically select the relevant part of the input.

Why softmax emphasizes larger scores

Softmax converts score differences into weight ratios. For two positions i and j,

$$\frac{\alpha_i}{\alpha_j} = \frac{\exp(s_i)}{\exp(s_j)} = \exp(s_i - s_j)$$

Therefore, if s_i is larger than s_j by 2, then $\alpha_i/\alpha_j = e^2 \approx 7.39$. This shows that even moderate score differences can create much larger attention differences.

In the example above, $s_{\text{animal}} - s_{\text{street}} = 2$, so

$$\frac{\alpha_{\text{animal}}}{\alpha_{\text{street}}} = e^2 \approx 7.39$$

This explains why attention strongly favors the token with the larger relevance score.

Why attention solves part of the RNN bottleneck

In a vanilla RNN, information from an early token must travel through many hidden states before influencing a later prediction. In attention, the later token can directly assign weight to the earlier

token. The path becomes shorter.

The RNN route is

$$h_1 \to h_2 \to \cdots \to h_T$$

The attention route is

$$q_T \to \alpha_i \to v_i$$

where token T can directly attend to token i. This is why attention is a major step toward modern Transformer-based models.

For the last few models, the conceptual transition is:

fragile recurrent memory $\longrightarrow$ gated memory $\longrightarrow$ vectorized language $\longrightarrow$ direct relevance-based tok

10 Transformer: Scaled Dot-Product Self-Attention

The Transformer replaces recurrence with self-attention. Instead of processing a sequence step by step, it processes all token positions at once. This is the key reason Transformers scale better than RNNs.

Let the input sequence be represented by a matrix $X \in \mathbb{R}^{T \times d}$, where T is the sequence length and d is the embedding dimension. Each row of X is the embedding of one token. The Transformer constructs three matrices:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

Here, Q is the query matrix, K is the key matrix, and V is the value matrix. The matrices W_Q, W_K, W_V are learned parameters.

The core operation is scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where d_k is the dimension of the key vectors.

Shape check

The matrix Q has shape $T \times d_k$, and K has shape $T \times d_k$. Therefore $QK^\top$ has shape $T \times T$. This matrix contains all pairwise attention scores between tokens.

The entry in row i and column j is $q_i^\top k_j$. It measures how much token i attends to token j. After applying softmax row by row, each row becomes an attention distribution over all token positions. Multiplying by V then forms a weighted average of value vectors for each token.

So the output has shape $T \times d_v$.

Proof: each Transformer attention row is a weighted average

For a fixed token position i, define the score from token i to token j as $s_{ij} = q_i^\top k_j/\sqrt{d_k}$. The attention weight is

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell=1}^{T} \exp(s_{i\ell})}$$

The output vector for token i is

$$o_i = \sum_{j=1}^{T} \alpha_{ij} v_j$$

Since $\exp(s_{ij}) > 0$, we have $\alpha_{ij} > 0$. Also,

$$\sum_{j=1}^{T} \alpha_{ij} = \sum_{j=1}^{T} \frac{\exp(s_{ij})}{\sum_{\ell=1}^{T} \exp(s_{i\ell})} = \frac{\sum_{j=1}^{T} \exp(s_{ij})}{\sum_{\ell=1}^{T} \exp(s_{i\ell})} = 1$$

Therefore, for each token i, the output o_i is a convex combination of value vectors $v_1, \ldots, v_T$. This proves that self-attention updates each token by taking a relevance-weighted average of all token representations.

Proof: why divide by $\sqrt{d_k}$

The scaling factor $\sqrt{d_k}$ is used to keep the attention scores from becoming too large.

Suppose the components of q and k are independent random variables with mean 0 and variance 1. Then the dot product is $q^\top k = \sum_{\ell=1}^{d_k} q_\ell k_\ell$.

For each term, $\mathbb{E}[q_\ell k_\ell] = \mathbb{E}[q_\ell]\mathbb{E}[k_\ell] = 0$. Also, since q_ℓ and k_ℓ are independent with variance 1, we have $\mathrm{Var}(q_\ell k_\ell) = \mathbb{E}[q_\ell^2 k_\ell^2] - 0 = 1$.

Therefore,

$$\mathrm{Var}(q^\top k) = \mathrm{Var}\left(\sum_{\ell=1}^{d_k} q_\ell k_\ell\right) = \sum_{\ell=1}^{d_k} \mathrm{Var}(q_\ell k_\ell) = d_k$$

So the typical size of $q^\top k$ grows like $\sqrt{d_k}$. If d_k is large, the raw dot products can become large in magnitude. Large softmax inputs make the softmax distribution extremely sharp, meaning one token gets almost all the weight and the gradients become small.

After scaling, we have

$$\mathrm{Var}\left(\frac{q^\top k}{\sqrt{d_k}}\right) = \frac{1}{d_k} \mathrm{Var}(q^\top k) = 1$$

Thus dividing by $\sqrt{d_k}$ normalizes the variance of the attention scores. This keeps the softmax more stable during training.

Numerical example

Suppose token i attends to three tokens with scaled scores

$$s_{i1} = 2, \qquad s_{i2} = 1, \qquad s_{i3} = 0$$

Then the softmax weights are

$$\alpha_{i1} = \frac{e^2}{e^2 + e^1 + e^0}, \qquad \alpha_{i2} = \frac{e^1}{e^2 + e^1 + e^0}, \qquad \alpha_{i3} = \frac{e^0}{e^2 + e^1 + e^0}$$

Since $e^2 \approx 7.39$, $e^1 \approx 2.72$, and $e^0 = 1$, the denominator is approximately 11.11. So $\alpha_{i1} \approx 0.665$, $\alpha_{i2} \approx 0.245$, and $\alpha_{i3} \approx 0.090$.

If the value vectors are v_1, v_2, v_3, then the output is $o_i \approx 0.665v_1 + 0.245v_2 + 0.090v_3$. Thus token i mainly uses information from token 1, but still keeps some information from tokens 2 and 3.

10.1 Multi-Head Attention

A single attention head computes one type of relevance. But language contains many relationships at once: nearby syntax, long-distance reference, subject-verb agreement, semantic similarity, and more. Multi-head attention lets the model learn several relevance patterns in parallel.

For head r, the model computes

$$\text{head}_r = \text{Attention}(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)})$$

Then the heads are concatenated and projected:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)W_O$$

Different heads can specialize. One head may focus on local grammar, another on previous mentions of a noun, and another on punctuation or sentence boundaries.

Why multi-head attention adds expressive power

One attention head produces one attention distribution per token. If there is only one head, token i must summarize all relevant relationships using one set of weights α_{ij}. But different linguistic relations may require different attention patterns.

With multiple heads, token i can have several distributions:

$$\alpha_{ij}^{(1)}, \alpha_{ij}^{(2)}, \ldots, \alpha_{ij}^{(H)}$$

So one head can attend to the subject, another to the verb, and another to a nearby modifier. The concatenated representation contains all these different weighted summaries. This makes multi-head attention more flexible than a single weighted average.

10.2 Positional Encoding

Self-attention alone does not know word order. If we permute the input tokens, the attention mechanism itself has no built-in recurrence telling it which token came first. Therefore, Transformers add positional information.

A common sinusoidal positional encoding is

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

and

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

The input becomes $X + PE$. This means each token representation includes both word identity and position.

Why this matters

Without positional encoding, self-attention is permutation-equivariant. That means if the input tokens are reordered, the outputs are reordered in the same way, but the mechanism does not know the original order. For language, this is a problem because "dog bites man" and "man bites dog" contain the same words but have different meanings.

Positional encoding fixes this by giving each token a location signal.

10.3 BERT: Masked Language Modeling and Bidirectional Pretraining

BERT uses a Transformer encoder. Its main pretraining objective is masked language modeling. The key idea is to hide some tokens and train the model to recover them using both left and right context.

Let the token sequence be $x = (x_1, x_2, \ldots, x_T)$. Let M be the set of masked positions. BERT observes the corrupted sequence $x_{\setminus M}$ and predicts the original tokens at the masked positions.

The masked language modeling objective is

$$\max_{\theta} \sum_{i \in M} \log P_\theta(x_i \mid x_{\setminus M})$$

Equivalently, the loss is

$$\mathcal{L}_{MLM}(\theta) = -\sum_{i \in M} \log P_\theta(x_i \mid x_{\setminus M})$$

Example

Suppose the original sentence is

"The capital of France is Paris."

BERT may receive

"The capital of France is [MASK]."

and it is trained to predict

Paris

The important point is that BERT can use both the left context, such as "The capital of France is," and the right context if the masked word appears in the middle of a sentence.

For example, in

"The student submitted the [MASK] before the deadline."

BERT can use both "submitted the" and "before the deadline" to infer that the masked word might be "assignment," "paper," or "report."

Proof: masked language modeling is a cross-entropy loss

For each masked position i, the true token is x_i. The model outputs a probability distribution $P_\theta(\cdot \mid x_{\setminus M})$ over the vocabulary. Let y_i be the one-hot vector for the true token. If the vocabulary has size V, the cross-entropy loss at position i is

$$-\sum_{v=1}^{V} y_{i,v} \log P_\theta(v \mid x_{\setminus M})$$

Since y_i is one-hot, only the true token has value 1. Therefore this reduces to

$$-\log P_\theta(x_i \mid x_{\setminus M})$$

Summing over all masked positions gives

$$\mathcal{L}_{MLM}(\theta) = -\sum_{i \in M} \log P_\theta(x_i \mid x_{\setminus M})$$

Thus the masked language modeling objective is exactly the cross-entropy loss for predicting masked tokens.

Why BERT is bidirectional

In a left-to-right language model, token x_i can only depend on $x_1, \ldots, x_{i-1}$. BERT instead predicts a masked token using the surrounding unmasked tokens. So the conditional probability has the form

$$P_\theta(x_i \mid x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_T)$$

This is bidirectional because the model can use tokens before and after the masked position.

Why bidirectionality helps

Consider the sentence:

"The bank was closed because of the heavy rain."

The word "bank" might mean a financial institution or the side of a river. The later words "was closed" strongly suggest the financial meaning. A left-to-right model at the word "bank" cannot see this later context. BERT-style bidirectional representation can use later words to refine the meaning.

This makes BERT especially useful for understanding tasks such as classification, named entity recognition, and question answering.

10.4 GPT: Autoregressive Next-Token Prediction

GPT uses a Transformer decoder. Its pretraining objective is autoregressive language modeling: predict the next token given previous tokens.

Given a sequence $x = (x_1, x_2, \ldots, x_T)$, the joint probability is factorized as

$$P_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} P_\theta(x_t \mid x_1, \ldots, x_{t-1})$$

The training objective is

$$\max_\theta \sum_{t=1}^{T} \log P_\theta(x_t \mid x_1, \ldots, x_{t-1})$$

Equivalently, the loss is

$$\mathcal{L}_{GPT}(\theta) = -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_1, \ldots, x_{t-1})$$

Proof: autoregressive factorization follows from the chain rule of probability

For any sequence $x_1, \ldots, x_T$, the chain rule of probability gives

$$P(x_1, \ldots, x_T) = P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_1, x_2) \cdots P(x_T \mid x_1, \ldots, x_{T-1})$$

This can be written compactly as

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

where the term for $t = 1$ is $P(x_1)$. This proves the autoregressive factorization used by GPT.

GPT models this factorization with a neural network. At each position t, it predicts x_t using only previous tokens.

Example

For the sentence

"The capital of France is Paris."

GPT learns probabilities such as

$$P(\text{capital} \mid \text{The})$$

$$P(\text{of} \mid \text{The capital})$$

$$P(\text{France} \mid \text{The capital of})$$

$$P(\text{Paris} \mid \text{The capital of France is})$$

This is why GPT is naturally suited to text generation. Generation is simply repeated next-token prediction.

Causal masking

GPT uses causal self-attention. Token i is allowed to attend only to tokens $j \leq i$. It cannot attend to future tokens $j > i$.

This is implemented by adding a mask M to the attention scores:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right) V$$

where

$$M_{ij} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

If $j > i$, then the score becomes $-\infty$. Since $e^{-\infty} = 0$, the softmax gives zero attention weight to future tokens. Therefore, token i cannot use future information.

Proof: causal masking blocks future tokens

For a fixed row i, the attention weight on token j is

$$\alpha_{ij} = \frac{\exp((q_i^\top k_j + M_{ij})/\sqrt{d_k})}{\sum_{\ell=1}^{T} \exp((q_i^\top k_\ell + M_{i\ell})/\sqrt{d_k})}$$

If $j > i$, then $M_{ij} = -\infty$. So the numerator becomes $\exp(-\infty) = 0$. Thus $\alpha_{ij} = 0$ for all $j > i$.

Therefore, causal masking forces every token to attend only to itself and earlier tokens.

10.5 BERT versus GPT

BERT and GPT are both Transformer-based, but they optimize different objectives.

Feature	BERT	GPT
Architecture	Transformer encoder	Transformer decoder
Direction	Bidirectional	Left-to-right
Objective	Masked token prediction	Next-token prediction
Main use	Representation and understanding	Generation and instruction following
Attention mask	Usually full attention	Causal attention

The conceptual difference is simple. BERT learns to fill in missing information using surrounding context. GPT learns to continue text from previous context.

BERT: recover missing token from both sides

GPT: predict next token from the left context

10.6 In-Context Learning

A major behavior of large GPT-style models is in-context learning. The model can often infer a task from examples placed in the prompt, without changing its parameters.

For example, the prompt may be:

English: cat → French: chat
English: dog → French: chien
English: house → French:

The model may continue with "maison." No gradient update occurs. Instead, the examples change the conditional distribution over the next token.

Mathematically, the model is still computing

$$P_\theta(x_{t+1} \mid x_1, \ldots, x_t)$$

but the prompt $x_1, \ldots, x_t$ now contains task examples. So the task is specified inside the context.

10.7 Scaling Laws: Why Scale Predictably Improves Performance

The Transformer made large-scale training practical. Scaling laws describe the empirical pattern that test loss improves predictably as model size, dataset size, and compute increase.

A stylized scaling law is

$$L(N, D, C) \approx L_\infty + aN^{-\alpha} + bD^{-\beta} + cC^{-\gamma}$$

where N is model size, D is dataset size, C is compute, L_∞ is the irreducible loss, and $\alpha, \beta, \gamma > 0$ are scaling exponents.

This formula says that increasing N, D, or C lowers loss, but with diminishing returns.

Proof: power laws become straight lines on log-log plots

Suppose the loss as a function of model size is

$$L(N) - L_\infty = aN^{-\alpha}$$

Taking logs on both sides gives

$$\log(L(N) - L_\infty) = \log a - \alpha \log N$$

This is a straight line in the variables $\log N$ and $\log(L(N) - L_\infty)$. The slope is $-\alpha$. Therefore, if model performance follows a power law, then plotting loss improvement against model size on a log-log scale produces an approximately linear relationship.

This is why scaling laws are often studied through log-log plots.

Numerical example

Suppose $L(N) - L_\infty = aN^{-0.1}$. If model size increases by a factor of 10, then the loss gap changes by

$$\frac{a(10N)^{-0.1}}{aN^{-0.1}} = 10^{-0.1} \approx 0.794$$

So a tenfold increase in model size reduces the reducible loss gap to about 79.4% of its previous value. This is an improvement, but not a proportional improvement. It shows diminishing returns.

If model size increases by a factor of 100, then

$$100^{-0.1} = 10^{-0.2} \approx 0.631$$

So a hundredfold increase reduces the reducible loss gap to about 63.1% of its previous value.

Compute tradeoff

Training compute is roughly proportional to the number of parameters times the number of training tokens. A stylized expression is $C \propto ND$, where N is parameter count and D is number of tokens.

For a fixed compute budget C, increasing N requires decreasing D, and increasing D requires decreasing N. So there is a tradeoff:

larger model with fewer tokens versus smaller model with more tokens

The statistical problem is to choose a good balance between model capacity and training data.

Example: fixed compute allocation

Suppose the compute budget allows $ND = 10^{18}$. If $N = 10^9$ parameters, then $D = 10^9$ tokens. If $N = 10^{10}$ parameters, then $D = 10^8$ tokens. The second model is larger, but it sees fewer tokens under the same compute budget.

This shows why simply increasing model size is not enough. A model can be too large relative to its data and become undertrained. A smaller model trained on more tokens may sometimes perform better at the same compute budget.

Why scaling laws matter

Scaling laws changed the way people think about AI progress. They suggest that performance improvement can be partly forecast from resource allocation. Modern LLM development therefore became a joint problem of architecture, data, model size, and compute.

The transition is:

$$\text{better algorithms} \quad \longrightarrow \quad \text{better algorithms plus predictable scaling}$$